ESP8266 Wi-Fi Robot Car

Complete explanation of the NodeMCU + L298N code, HTTP commands, PWM speed control, debugging, and safety

Purpose of this guide: Explain the original program and the modified version, show how a phone command becomes motor movement, and help diagnose why a car may connect to Wi-Fi but not respond.

> **Important:** The code discussed here is the modified version from the conversation. Its behavior still depends on the mobile app sending HTTP requests with the expected `State` parameter and on the motor driver being wired correctly.

1. What the project does

This project controls a two-motor robot car using an ESP8266 NodeMCU board and an L298N dual H-bridge motor driver. The NodeMCU creates its own Wi-Fi access point named `NodeMCU Car`. A phone connects to that network and sends HTTP requests such as `/?State=F`. The ESP8266 reads the command, chooses a movement function, and sets the L298N direction and enable signals.

```
Phone / control app
       |
       | Wi-Fi + HTTP request, e.g. /?State=F
       v
ESP8266 NodeMCU (access point + web server)
       |
       | GPIO direction signals + PWM enable signals
       v
L298N dual H-bridge driver
       |                          |
       v                          v
 Right DC motor              Left DC motor
```

The phone does not directly power or drive the motors. It sends a small text command. The NodeMCU interprets that command, and the L298N switches motor current from a suitable motor power supply.

2. Hardware and signal flow

2.1 ESP8266 NodeMCU

The ESP8266 is the controller. It creates the Wi-Fi network, runs the HTTP server, reads incoming commands, and sets the GPIO outputs. Its GPIO pins are logic-level control signals; they should not power the motors directly.

2.2 L298N motor driver

The L298N contains two H-bridge channels. Each channel can drive one brushed DC motor in either direction. The IN pins choose direction; ENA and ENB enable the channels and can receive PWM to regulate average motor drive.

2.3 Power and common ground

Use a suitable motor supply for the motors and driver. Connect the NodeMCU ground and L298N ground together so the control signals share a reference. Do not power motors from the NodeMCU 3.3 V pin. Check your particular L298N module's jumper and supply arrangements before wiring.

> The L298N has a voltage drop and can heat up. Make sure the motor current is within the driver's limits and the supply can provide the required current.

3. Pin definitions

Code name	GPIO	NodeMCU label	Function in this code
ENA	14	D5	Right motor PWM / enable
ENB	12	D6	Left motor PWM / enable
IN_1	15	D8	Right motor direction input 1
IN_2	13	D7	Right motor direction input 2
IN_3	2	D4	Left motor direction input 1
IN_4	0	D3	Left motor direction input 2

Boot-pin caution: GPIO0 (D3), GPIO2 (D4), and GPIO15 (D8) are ESP8266 boot-strapping pins. External circuitry must not force them to incorrect levels during startup. If the board sometimes fails to boot, review these pin choices and the driver module's electrical connections.

4. The libraries and global variables

```
#include <ESP8266WiFi.h>
#include <WiFiClient.h>
#include <ESP8266WebServer.h>

String command;
int speedCar = 800;
int speed_Coeff = 3;

const char* ssid = "NodeMCU Car";
ESP8266WebServer server(80);
```

ESP8266WiFi.h provides Wi-Fi functions, including access-point mode. **WiFiClient.h** provides client networking support; it is included here although the shown code does not directly create a WiFiClient object. **ESP8266WebServer.h** provides the HTTP server.

- `command` stores the received text command, such as F or S.
- `speedCar = 800` is the initial PWM setting.
- `speed_Coeff = 3` divides one motor's PWM in diagonal movement functions.
- `ssid` is the Wi-Fi network name.
- `ESP8266WebServer server(80)` creates a web server listening on HTTP port 80.

5. setup(): startup sequence

```
void setup() {
  Serial.begin(115200);
  // Configure motor pins as outputs
  // Stop the motors
  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid);
  IPAddress myIP = WiFi.softAPIP();
  Serial.println(myIP);
  server.on("/", HTTP_handleRoot);
  server.onNotFound(HTTP_handleRoot);
  server.begin();
}
```

At startup, the program opens serial communication at 115200 baud, configures the motor pins as outputs, and should put the motor outputs into a stopped state. It then sets the ESP8266 to access-point mode and creates the network named NodeMCU Car. `WiFi.softAPIP()` obtains the access point's IP address; the default is commonly 192.168.4.1. Finally, it registers the HTTP handler and starts the server.

The code uses `server.on("/", HTTP_handleRoot)` for the root path. The `server.onNotFound(HTTP_handleRoot)` line also routes otherwise-unmatched paths to the same handler. This can help with simple testing, but a dedicated not-found handler is often clearer in a larger project.

6. How phone commands reach the NodeMCU

A command URL is an HTTP GET request. For example:

```
http://192.168.4.1/?State=F
```

Here, `192.168.4.1` is the ESP8266 access-point address, `/` is the request path, and `State=F` is a query parameter. The parameter name must match the code exactly: `State`.

The program calls `server.handleClient()` in `loop()`. This lets the web-server library handle incoming HTTP requests. The registered handler runs for a matching request. Inside that handler, `server.hasArg("State")` checks whether the parameter exists, and `server.arg("State")` retrieves its value.

> **Critical diagnosis:** Wi-Fi association only proves that the phone joined the access point. It does not prove that the app is sending HTTP requests, that the URL is correct, or that it uses the `State` parameter. A browser test is useful to separate app issues from server issues.

7. Command table

Command	Action	Meaning
F	goAhead()	Both motors forward
B	goBack()	Both motors reverse
L	goLeft()	Motors in opposite directions to pivot left
R	goRight()	Motors in opposite directions to pivot right
I	goAheadRight()	Forward curve, one motor slower
G	goAheadLeft()	Forward curve, other motor slower
J	goBackRight()	Reverse curve, one motor slower
H	goBackLeft()	Reverse curve, other motor slower
S	stopRobot()	Stop command
0–9	setSpeed(...)	Select PWM speed setting

The physical turning direction depends on the motors' mounting and wiring. If the car turns the wrong way, check motor polarity and whether the left/right channels match the code comments.

8. How the motor direction functions work

8.1 Forward

```
digitalWrite(IN_1, LOW);
digitalWrite(IN_2, HIGH);
digitalWrite(IN_3, LOW);
digitalWrite(IN_4, HIGH);
analogWrite(ENA, speedCar);
analogWrite(ENB, speedCar);
```

For each motor, the two IN pins are set to opposite logic levels. This selects one direction through the H-bridge. Both enable pins receive the current PWM setting, so both motors are commanded to run at the same drive level.

8.2 Reverse

The reverse function swaps the HIGH and LOW values for both motor channels. This reverses the voltage polarity applied by the H-bridge to each motor, reversing rotation.

8.3 Pivot turns

The left and right turn functions command the motors in opposite directions. If the motors are mounted conventionally, one wheel pushes forward while the other pushes backward, causing the chassis to rotate around a point near its center.

8.4 Curved movement

```
analogWrite(ENA, speedCar / speed_Coeff);
analogWrite(ENB, speedCar);
```

A diagonal function keeps both motors moving in the same direction but reduces one motor's PWM. With `speedCar = 900` and `speed_Coeff = 3`, the reduced PWM is 300. The speed ratio is only approximate because motor speed is not perfectly proportional to PWM and depends on load, battery voltage, friction, and driver losses.

9. PWM and speed settings

PWM stands for Pulse Width Modulation. The controller rapidly switches an enable signal on and off. Changing the duty cycle changes the average drive applied to the motor. In the ESP8266 Arduino core, the default PWM range is normally 0 to 1023.

App command	PWM value	Approx. fraction of 1023
0	400	39%
1	470	46%
2	540	53%
3	610	60%
4	680	66%
5	750	73%
6	820	80%
7	890	87%
8	960	94%
9	1023	100%

These are PWM command levels, not guaranteed RPM values. A motor may not start at low PWM because it needs enough torque to overcome static friction. The L298N also loses voltage, so the motor's actual terminal voltage is lower than the supply voltage.

10. What changed from the original code

A. Command processing moved into HTTP_handleRoot()

In the original code, the handler printed the State parameter, while loop() separately attempted to read server.arg("State"). The modified code reads the parameter and executes the corresponding movement function in the request handler. This keeps the request's data and its processing together.

B. More Serial Monitor messages

The modified handler prints that a request arrived, the request URI, and the received command. These messages show whether the HTTP request reached the ESP8266 and whether State was extracted.

C. Explicit response messages

The modified code returns text such as 'Command received: F'. This is useful during browser testing. An HTTP 200 response confirms the handler responded; it does not prove the motors physically moved.

D. Explicit stop PWM

The stop function now sets ENA and ENB PWM to zero. In the original function, both direction inputs were LOW but the enable PWM remained nonzero. The original inputs already commanded a stopped state on a typical L298N channel, but zero PWM makes the intention explicit.

E. Movement timeout

The modified code records the time of each movement command and stops the motors after one second without another movement command. This can reduce the chance of a car continuing after communication stops.

F. Speed helper

setSpeed() updates the speed variable and, if the car is moving, updates both enable PWM outputs. This makes a speed change take effect immediately, although it does not preserve different PWM levels for diagonal movement until another movement command is issued.

11. Important limitation in the modified code

The timeout is refreshed by movement functions, not by speed-setting commands. The app must repeatedly send a movement command while the button is held, often every 100–300 ms, for a one-second timeout to allow continuous movement. If the app sends only one command per button press, the car stops after one second.

> Also, setSpeed() sets ENA and ENB to the same PWM whenever motorsMoving is true. If speed is changed while a diagonal function is active, this temporarily removes the left/right PWM difference. A more complete implementation would remember the active movement mode and recalculate the PWM for each motor.

The original and modified code both rely on the mobile client sending HTTP GET requests with a query parameter named State. If the app uses a different path, a different parameter name, POST body data, or WebSockets, this handler will not process it as expected.

12. How to test the server independently of the app

Test 1 — Check startup

Upload the sketch, open Serial Monitor at 115200 baud, and reset the NodeMCU. Look for the Wi-Fi name, IP address, and 'HTTP Server Started'.

Test 2 — Join the access point

Connect the phone to NodeMCU Car. If the phone warns that the network has no internet, choose to stay connected. Mobile operating systems may otherwise switch back to mobile data or another Wi-Fi network.

Test 3 — Send a browser command

```
http://192.168.4.1/?State=S
http://192.168.4.1/?State=F
http://192.168.4.1/?State=B
```

Start with State=S. Raise the wheels before testing F or B. The browser should show a response and the Serial Monitor should show the corresponding command.

Test 4 — Interpret the result

Observation	Likely next check
No Serial output	Confirm phone is connected to the NodeMCU AP, IP is correct, browser request is sent, and server sta
HTTP request, but State missing	Check exact URL and parameter spelling: ?State=F.
Command received: F, but no motor movement	Check motor supply, common ground, ENA/ENB jumpers or wiring, motor outputs, and pin mapping.

Observation	Likely next check
Motors run but direction is wrong	Check motor wire polarity and which physical motor is connected to each channel.
Browser test works, app fails	Inspect the app's URL, HTTP method, parameter name, and whether it repeats commands while a butt

13. Safety and reliability notes

• Test with the wheels raised and keep hands, wires, and loose clothing away from the wheels.

• Use an appropriate battery and motor supply; do not power motors from the NodeMCU 3.3 V pin.

• Connect grounds together and check the L298N module's ENA/ENB jumpers. If PWM control is used on an enable pin, the wiring must allow the ESP8266 to control that pin.

• ESP8266 GPIO pins are 3.3 V logic. Do not apply 5 V directly to its GPIO pins.

• GPIO0, GPIO2, and GPIO15 affect boot mode. The attached driver circuit must not force the wrong startup levels.

• The timeout is not an emergency stop. It depends on the controller continuing to run and on the motor driver responding correctly.

• The stop function commands the driver to stop driving; the car can still coast due to inertia.

14. Quick glossary

Term	Simple meaning
GPIO	A microcontroller input/output pin.
HTTP	A request-response protocol used by the phone and web server.
Access point (AP)	A device that creates a Wi-Fi network for other devices to join.
Query parameter	A name/value pair in a URL, e.g. State=F.
H-bridge	A circuit that can reverse voltage across a motor to reverse its direction.
PWM	Rapid ON/OFF switching used to adjust average motor drive.
Baud rate	Serial communication speed; here 115200.
Timeout	A rule that triggers an action if no expected command arrives in time.

15. Final mental model

Think of the project as three separate stages: (1) the phone sends a correctly formatted HTTP command, (2) the NodeMCU reads the command and sets GPIO/PWM outputs, and (3) the L298N and power supply deliver the required current to the motors. Wi-Fi connection confirms only the first network step; it does not confirm that the app sent a valid command or that the motor circuit works.

Best debugging order: browser request → Serial Monitor confirms command → check GPIO/control signals → check L298N and motor power → finally test the mobile app.